



Devin: The AI Software Engineer

Agenda

01 Cognition Overview

30,000-foot view of the company, products, and how we serve customers

02 Market Environment & Challenges

Why software delivery is the bottleneck, and why now

03 Meet Devin!

Architecture, workflow, security model, and live examples

04 Customer Impact

Stories and measurable outcomes

05 Next Steps

How to get started?

Our understanding of Shopify

Your business

- Commerce OS for millions of merchants in 175+ countries; ~\$300B annual GMV, ~\$9B revenue, growing 25%+.
- Every storefront renders through Liquid; every merchant depends on your platform's uptime and speed.
- Moving up-market (Enterprise, Commerce Components) while defending the SMB core.

Your objectives

- "Reflexive AI usage" mandate: prove AI cannot do the job before asking for headcount.
- Ship faster on the world's largest Rails codebase while keeping quality and performance.
- Protect merchant trust: security, uptime, and safe rendering of untrusted templates.
- Scale the developer ecosystem (themes, apps) without scaling support load.

How Cognition can help

- Devin turns the AI mandate into merged PRs, not just faster typing: migrations, tests, bugs, and upgrades done end to end.
- Devin Wiki and Review compound knowledge and quality across thousands of engineers and repos.
- Enterprise controls (VPC, SSO, audit logs, no training on code) fit a platform millions of merchants trust.

Questions for the room

- CTO: since the AI mandate, what share of PRs are AI-assisted, and how has merge time or defect rate changed?
- CTO / VP Eng: what is the biggest chunk of engineering time that is not roadmap right now?
- CISO: what did you need to see before approving Cursor and Copilot for engineers? Would an autonomous agent be judged differently?
- Business: which merchant-facing initiative slipped last quarter because engineering was the constraint?

Where Devin moves Shopify's initiatives

Shopify initiative	Engineering work it creates	What Devin does	Business outcome
Reflexive AI usage; headcount only where AI cannot do the job	Backlog of well-specified tickets, tests, and upgrades nobody has time for	Sessions: takes tickets from Slack/GitHub to tested, reviewed PRs, many in parallel	Mandate becomes measurable: PRs merged and hours reclaimed per team
Storefront performance and reliability (Liquid, themes, checkout)	Perf regressions, edge-case bugs, filter/tag features requested by theme devs	Sessions + Review: reproduce, fix, benchmark; Review catches regressions on every PR	Faster storefronts, fewer merchant-facing incidents, higher conversion
Largest Rails monolith in the world: keep it modern	Ruby/Rails upgrades, gem CVEs, deprecations across thousands of modules	Automations: scheduled dependency and CVE sweeps that open PRs automatically	Security debt shrinks continuously instead of in painful annual pushes
Grow the theme and app developer ecosystem	Docs drift; partners and new hires ask the same questions of senior engineers	Wiki + Ask Devin: always-current architecture docs and codebase Q&A	Faster onboarding, fewer interrupts for senior engineers, better partner experience
Merchant trust: security and compliance at scale	Every automated actor needs auditability, scoped access, and data guarantees	Security: VPC option, SSO/SCIM, encrypted secrets, immutable session logs, no training on code	AI adoption the CISO can sign off on

Cognition - the applied AI lab behind Devin

Mission: build collaborative AI teammates that expand what engineering teams can ship.



Market Environment & Challenges

Engineering is every business's bottleneck

~40%

of developer time is spent on maintenance and toil rather than building (*McKinsey*)

\$85B+

estimated annual cost of technical debt to enterprises

Months

to onboard a new engineer to a large legacy codebase

Weeks

median time known CVEs stay unpatched in production dependencies

-
- Every company is now a software company, but headcount growth cannot match demand for software.
 - Legacy systems, migrations, and compliance work consume the most experienced engineers.
 - Security teams inherit vulnerability debt they cannot fix themselves.

Enterprises need a multiplier on engineer capability and time allocation

What we hear from engineering leaders

Reactive

- 40-60% of engineering time goes to maintenance, migrations, and toil, not roadmap.

Execution bottleneck

- Teams are the constraint by headcount
- Headcount constraints are real
- Team is stuck doing repetitive, low-value work.

Context Switching

- Meetings & interruptions hinder deep productivity

Technical debt

- Compounds & delayed refactoring makes systems harder to modify
- Bandwidth limitations → Dependency upgrades and CVE patches are 'known and deferred'

Security & Compliance

- Essential but time consuming
- Engineers already paste code into consumer AI tools.
- Security has limited visibility or control.

Quality vs Speed

- Pressure to ship
- Keep pace with revenue demands
- Copilots help type faster but do not take work off the board.

- Legacy systems slow shipping velocity

Copilot → Colleague

AI Copilot (Gen 1)

- Suggests the next line inside the editor
- Requires a human at the keyboard for every step
- No environment: cannot run tests or fix CI
- Productivity gain: ~10-20% per developer
- Backlog size unchanged

AI Software Engineer (Gen 2)

- Takes a ticket and returns a tested pull request
- Works asynchronously and in parallel: many agents at once
- Own sandboxed machine: shell, browser, CI, deployment
- Human shifts to reviewer and architect
- Backlog actually shrinks

Demo repo: Shopify/liquid

Liquid is the open-source template engine behind every Shopify storefront theme, used by millions of merchants and the entire theme-developer ecosystem.

It is small enough to understand in a 40-minute meeting (~7k lines of Ruby, strong test suite, active CI) and important enough that every change matters: a bug or a slow filter shows up on real storefronts.

It is also a security-sensitive codebase by design: Liquid exists so merchants can edit templates without running arbitrary code on Shopify's servers. That makes it the right place to talk about safe, reviewed, auditable change.

~7k

lines of Ruby, 60+ standard filters

Millions

of storefronts render Liquid

Active

CI on GitHub Actions,
maintained by Shopify

Ruby 3.x

maintained by the Rails-
heaviest company on earth

github.com/Shopify/liquid | In the demo, Devin works on a fork with the same code, tests, and CI.

Live demo: a day in the life of a Liquid maintainer

1 Wiki	2 Session	3 Review	4 Automation	5 Security
Onboard in minutes Open the Devin Wiki for Shopify/liquid: architecture, render pipeline, how filters are registered. Ask Devin: 'Where would I add a new standard filter and what tests must change?'	Ticket to PR From Slack: '@Devin add an `average` filter alongside `sum`, with tests, docs and a History.md entry.' Devin plans, edits, runs the suite, opens a PR.	Quality gate Devin Review comments on the PR: correctness, edge cases (nil, non-numeric), style vs. repo conventions. Show a finding and the fix.	Keep it modern Show a scheduled automation: weekly gem/CVE sweep on the repo that opens PRs and posts a Slack summary.	CISO view Settings tour: SSO, repo scoping, secrets vault, session audit log, VPC option, data-retention and no-training policy.
Ties to: Ecosystem & onboarding	Ties to: AI mandate; storefront features	Ties to: Storefront reliability	Ties to: Rails monolith; security debt	Ties to: Merchant trust

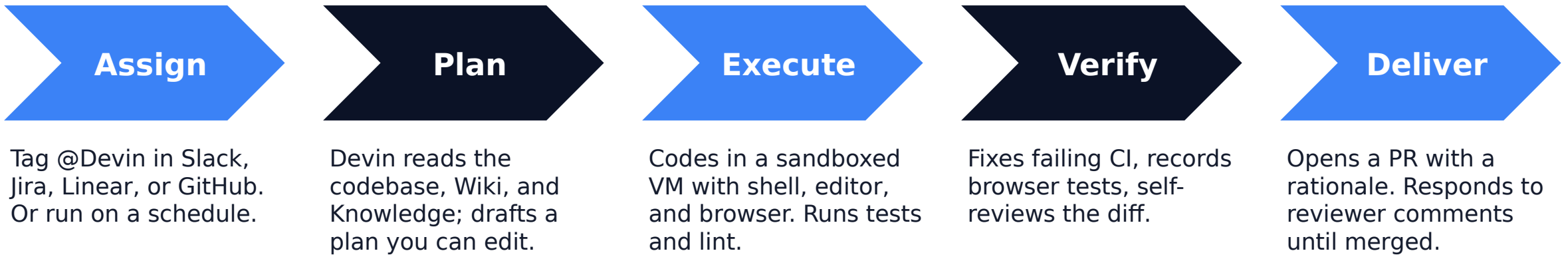
Integrations shown throughout: Slack trigger, GitHub PR, CI status. Backup: pre-recorded session if live run is slow.

SECTION 02

Meet Devin

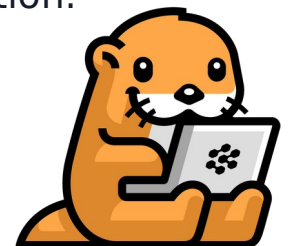


Ticket → Merged PR



Humans stay in control

- Devin never merges its own code; every change goes through your normal PR review and branch protection.
- Full session transcript: every command, file edit, and browser action is logged and replayable.
- Interrupt, redirect, or take over the session at any time.



One platform, two ways to work with AI



Devin | Autonomous cloud agent

- Delegate whole tasks: 'Upgrade this service to Java 21 and open a PR.'
- Runs in its own sandboxed machine with shell, editor, and browser.
- Plans, codes, tests, fixes CI, and responds to review comments.
- Triggered from Slack, Jira, Linear, GitHub, or the API.

Devin Desktop | Agentic IDE

- Where engineers work interactively, with deep codebase awareness.
- Cascade agent for multi-file edits in the editor.
- Hand off to Devin when a task is ready to run unattended.
- Agent command center

Supporting capabilities

Devin Review

AI code review on every PR: bugs, security, style

Devin Wiki & DeepWiki

Auto-generated, always-current codebase documentation

Devin CLI

AI Software Engineer in your terminal, with every frontier model

Playbooks & Automations

Reusable procedures, scheduled and event-driven runs

Built to understand large, messy codebases

Codebase Intelligence

- Devin Wiki indexes every repo into living architecture docs.
- Devin Search answers 'where is X handled?' across services.
- Knowledge: your conventions, gotchas, and runbooks, learned once and reused.

Reproducible Environments

- Each session runs on an isolated VM built from a versioned blueprint (deps, services, secrets).
- Snapshots make startup fast and deterministic.
- Devin can browse, run services, and drive a real browser to verify UI.

Scale & Orchestration

- Run dozens of Devins in parallel on a migration backlog.
- Playbooks encode repeatable procedures; Automations trigger on schedules or events.
- API and Slack-native: fits existing workflows, no new UI to learn.

Where Devin delivers first

Migrations & upgrades

Language/framework version bumps, Python 2 to 3, Angular to React, cloud re-platforming. Repetitive, well-specified, and enormous.

Test coverage & flaky CI

Backfill unit tests, fix flaky suites, keep coverage gates green while humans build features.

Bug fixes & on-call toil

Triage Sentry/Datadog alerts, reproduce, fix, and open PRs before the on-call engineer wakes up.

Security & dependency debt

Patch CVEs across hundreds of repos, rotate deprecated APIs, enforce lint and policy.

Codebase Q&A & onboarding

New hires ask Devin instead of interrupting seniors. Docs stay current automatically.

Internal tools & integrations

Build the internal dashboards, scripts, and glue code that never make the roadmap.

Devin vs. Copilot, Cursor, and Claude Code

	Copilot	Cursor	Claude Code	Devin
Where it runs	In the IDE	In the IDE (fork of VS Code)	In your terminal	Its own cloud VM per task; parallel
Who drives	Developer, keystroke by keystroke	Developer, prompt by prompt	Developer, in a live terminal	Devin, end to end; humans review
Unit of work	Line / function	File / multi-file edit	Task in a local session	Ticket to merged PR, with CI
Runs tests, fixes CI, drives a browser	No	Partly (agent mode)	Yes, on the dev's machine	Yes, in an isolated sandbox with full audit log
Org knowledge	Per-user context	Per-repo rules	CLAUDE.md per repo	Wiki, Knowledge, Playbooks shared org-wide
Runs unattended (schedules, events)	No	No	Scriptable by you	Automations built in; Slack/Jira/GitHub triggers
Security posture	GitHub enterprise controls	Privacy mode, SOC 2	Runs with the dev's local access	VPC option, SSO/SCIM, scoped GitHub App, secrets vault, SOC 2

Position: keep Cursor/Copilot for the engineer in the loop. Devin is for the work you want off the engineer's plate.

How we serve customers

1. Scope

Identify 2-3 high-volume, well-specified task types (migrations, test coverage, bug fixes).

2. Connect

Integrate with GitHub/GitLab, Slack, Jira. Configure Devin's environment and Knowledge.

3. Pilot

Run 4-8 weeks with a champion team. Measure PRs merged, cycle time, hours saved.

4. Expand

Roll out playbooks and automations org-wide. Enterprise controls, SSO, VPC.

What you get from Cognition

- Dedicated forward-deployed engineering during onboarding; we learn your codebase alongside Devin.
- Success metrics agreed up front and reported throughout the pilot.
- Enterprise support, security documentation, and a shared roadmap channel.

SECTION 05

Customer Impact

Stories and measurable outcomes

Nubank: a multi-year migration in months

Challenge

One of the world's largest digital banks needed to migrate a critical ETL codebase spanning millions of lines. Estimated at 1,000+ engineer-weeks of repetitive, high-risk work.

Approach

Cognition's team and Nubank engineers ran many Devins in parallel against the migration, with humans reviewing and merging every PR.

Outcome

Migration completed in a fraction of the projected time, with human engineers redeployed to customer-facing work.

12x

efficiency improvement vs.
manual migration

20x

cost savings on the
migration

MLOC

millions of lines of code
migrated

Months

not years, to completion

Impact across industries

Goldman Sachs

Global bank

Piloting Devin as an AI engineer alongside its ~12,000 developers, focused on legacy code modernization and augmenting developer productivity under strict governance.

Ramp

Fintech

Uses Devin for backlog tasks and internal tooling, freeing engineers to focus on product velocity in a fast-scaling environment.

MongoDB

Developer platform

Adopted Devin to accelerate engineering work across a large, distributed codebase, with Devin handling well-scoped tasks end to end.

Bilt

Consumer fintech

Leverages Devin to expand engineering capacity without proportional headcount growth.

The business case

Lever	How to measure	Illustrative impact
Capacity reclaimed	Engineer-hours on tasks completed by Devin	Team of 50 x 20% toil = 10 FTE-equivalents
Cycle time	Ticket-to-merge time for Devin-eligible tasks	Days to hours; parallel execution
Migration cost	Cost per repo/service migrated	10-20x reduction (Nubank)
Security posture	Mean time to patch known CVEs	Weeks to days, org-wide
Onboarding	Time to first meaningful PR for new hires	Weeks shaved via Wiki and Ask Devin

Pilot success criteria are set jointly against these levers and reviewed weekly.

SECTION 06

Next Steps

A proposal to prove value in your environment

Proposal: a 6-week scoped pilot

Week 0-1 | Security & setup

- Security review: SOC 2 report, DPA, architecture Q&A
- Connect repos, Slack, issue tracker
- Define 2-3 target task types and success metrics

Week 2-5 | Run

- Champion team assigns real backlog tickets to Devin
- Cognition forward-deployed engineer embedded
- Weekly metrics: PRs merged, cycle time, hours saved

Week 6 | Decide

- Executive readout vs. agreed criteria
- ROI model with your actual numbers
- Rollout plan and commercial proposal

What we ask of you

One executive sponsor, one champion team (5-10 engineers), a security point of contact, and a backlog of real tickets. We bring the rest.

Give your engineers a teammate. Give your roadmap a chance.

Devin takes on the migrations, tests, and toil that consume your best people, inside the security boundaries your CISO defines, with every change reviewed by your team.

Proposed next step

Security review kickoff and pilot scoping session within the next two weeks.

SECTION 06

Appendix

Where does engineering time actually go?

Delivery & Velocity

- What share of engineering capacity goes to new features vs. maintenance, migrations, and tech debt?
- Which roadmap items slipped last quarter, and why?
- How long does a typical PR take from open to merge? Where does it stall?
- What is the ratio of senior to junior engineers, and where are seniors spending review time?

Backlog & Scale

- What is sitting in the backlog that never gets prioritized: version upgrades, test coverage, flaky CI, documentation?
- Are there large migrations (language, framework, cloud, monolith-to-services) on the horizon?
- How many repos and services does a new hire need to understand to be productive?
- If you had 30% more engineering capacity tomorrow, what would you point it at?

What does 'safe adoption' look like to you?

Governance & Risk

- Are engineers already using AI coding tools? Sanctioned or shadow?
- What is your policy on source code leaving your environment or being used for model training?
- Which compliance frameworks matter most: SOC 2, ISO 27001, HIPAA, PCI, FedRAMP?
- How do you audit who or what changed production code today?

Controls & Vulnerability Debt

- How do you manage secrets and access for automated systems (CI, bots, service accounts)?
- How long do known CVEs in dependencies sit open before they are patched?
- Would you require VPC or self-hosted deployment, SSO, and audit logs for an AI agent?
- What would an AI agent need to prove before you would allow it to open a pull request?

Security and trust, designed for the CISO

Data Protection

- Your code is not used to train models.
- Encryption in transit and at rest.
- Secrets stored in an encrypted vault and injected at runtime; never exposed in transcripts.
- Configurable data retention.

Deployment & Access

- Multi-tenant SaaS, or Enterprise VPC deployment in your cloud account.
- SSO/SAML, SCIM, role-based access, repo-level scoping.
- Devin authenticates as a scoped GitHub App, not a personal token.

Compliance & Audit

- SOC 2 Type II attested.
- Complete, immutable session logs for every action taken.
- No self-merge; branch protection and CODEOWNERS enforced.
- Devin Review and code scans surface vulnerabilities in existing code.

Appendix: objections and responses

"We already have Copilot."

Great, keep it. Copilot helps engineers type; Devin takes tickets off the board. Customers run both; the value is additive.

"Our code can't leave our environment."

Enterprise VPC deployment runs in your cloud account. No training on your code, encrypted secrets, full audit logs.

"How do we measure it?"

Jointly defined pilot metrics: PRs merged, cycle time, hours saved, CVEs patched. Reported weekly.

"AI code isn't trustworthy."

Neither is any unreviewed code. Devin runs tests, fixes CI, and never self-merges. Your review process is unchanged; the volume of reviewable work increases.

"Will this replace engineers?"

It replaces toil. Customers redeploy engineers to product work and report higher retention when migrations and maintenance are automated.

"What does it cost?"

Consumption-based (ACUs) with team and enterprise tiers. Pilot pricing is scoped to the agreed use cases; we model ROI on your numbers.